

SENSORFUSION-HAR: A RESERVOIR-AUGMENTED MULTI-PATH NEURAL NETWORK FOR ON-DEVICE HUMAN ACTIVITY RECOGNITION ON THE ESP32

Ujjwal Manot

E23ECEU0016

Mahi Jerry

E23ECEU0001

under the supervision of:

Dr. Rohan Katti

Assistant Professor



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

SCHOOL OF ENGINEERING AND APPLIED SCIENCES

BENNETT UNIVERSITY

GREATER NOIDA, UTTAR PRADESH, INDIA

DECEMBER 2025

Table of Contents

1	Introduction	4
1.1	Objectives of the Project	4
2	System Block Diagram	6
2.1	Related Work	6
2.2	Dataset Preparation and Harmonisation	7
2.3	Model Architecture	7
2.3.1	Trainable Echo-State Reservoir	7
2.3.2	Depthwise-Separable Convolutional Encoder	8
2.3.3	Multi-Scale Temporal Block	8
2.3.4	Spectral Gated Fusion	8
2.3.5	Patch Micro-Attention	8
2.3.6	Orientation Micro-Path	8
2.3.7	Scaled Binary Classifier	9
2.4	Novel Research Contributions	9
2.5	Self-Supervised Pre-Training	10
2.6	Training Protocol	10
3	Results	12
3.1	Why Some Classes are Harder than Others	12
3.2	Comparison with the V1 Baseline	14
3.3	Memory Budget on the ESP32	14
3.4	Edge Deployment Pipeline	14
4	Conclusion and Future work	17
5	References	18

CERTIFICATE

This is to certify that **Ujjwal Manot** bearing enrollment number **E23ECEU0016** and **Mahi Jerry** bearing enrollment number **E23ECEU0001** have carried out this sixth semester Hardware Design Project for the course EECE308J titled “**Design and Implementation of an On-Device Human Activity Recognition System on the ESP32**” under the supervision of Dr. Rohan Katti, Assistant Professor, Department of Electronics and Communication Engineering, School of Engineering and Applied Sciences, Bennett University. The performance of the students is satisfactory.

Signature of the mentor

Dr. Rohan Katti
Assistant Professor
Department of ECE

ABSTRACT

This report describes SensorFusion-HAR, a small deep neural network for human activity recognition that runs on a stock ESP32 microcontroller without any network connection. The classifier consumes one-second windows of six-axis IMU data sampled at 50 Hz (each window shaped 50×6) and produces a posterior over eleven activities: walking, sitting, standing, lying down, going up stairs, going down stairs, jogging, jumping, cycling, running, and waist bending. The pipeline is built from five sequential stages and one parallel orientation path. A trainable echo-state reservoir, whose spectral radius is learned through a sigmoid reparameterisation, projects the raw window into a higher-dimensional dynamical state. Two depthwise-separable convolution blocks compress that state along time. A multi-scale temporal block, with kernels of 3, 5 and 9 samples in parallel, captures rhythms that range from heel-strikes to slow torso bends. A spectral gated fusion module mixes the reservoir trace and the convolutional features in two frequency sub-bands, and a patch-based micro-attention head attends across five non-overlapping patches of the result. A separate orientation micro-path computes 27 statistical descriptors and is concatenated with the attention output before the classification head, which itself is binarised with a per-output learnable scale and a straight-through estimator.

The encoder is pre-trained on three publicly available datasets (UCI-HAR, PAMAP2, and MHEALTH) using a masked sensor modelling objective, and is then frozen during a focal-loss head-training phase that uses reservoir-manifold mixup for boundary smoothing. The final model has 75,290 parameters, reaches 89.36% accuracy and 0.9028 macro F1 on a held-out partition, and stays above 0.83 F1 on every class. After INT8 post-training quantisation the model occupies roughly 74 KB of flash and uses about 40 KB of SRAM at inference, which leaves the remaining 470 KB of the 512 KB on-chip memory for the application code. The complete system, including a phone-streaming dashboard for live demonstration and a hardware-in-the-loop validation pipeline that drives the firmware with controlled dataset windows over UART, is released alongside this report.

1 Introduction

Continuous human activity recognition from wearable inertial sensors sits behind a long list of applications: step counting, fall detection, sleep staging, cardiac rehabilitation, sports analytics, and increasingly, glucose-management algorithms that need to know when their wearer is exercising [1, 2]. The simplest version of the problem is well understood. Given a short sliding window of accelerometer and gyroscope readings, classify which of a small set of activities the user is doing. The harder version, and the one tackled in this project, is to do that on a microcontroller. No cloud round-trip, no offloading to a phone, no Wi-Fi handshake. Everything happens on the chip, in real time, on a battery.

The reason this matters is partly privacy and partly cost. A wearable that ships motion data to a server requires consent that few people give and a backend that someone has to pay for. A wearable that decides on the chip ships only the label, which is a few bytes per second and easy to transmit over BLE when a phone happens to be nearby. The harder challenge is the chip itself. The ESP32 used here is a dual-core Xtensa LX7 running at 240 MHz with 512 KB of on-chip SRAM and no neural-network accelerator. There is no GPU, no DSP, no NPU. Every multiply-accumulate happens in the integer ALU. A model that runs comfortably on a phone in a few milliseconds can take seconds on the ESP32, or simply not fit at all.

Three constraints shape the design. First, the model has to be small. The target is around 100 KB of INT8-quantised weight memory, which leaves enough headroom for the TFLite Micro runtime, normalisation buffers, the I2C driver, and a small task scheduler. Second, the model has to be fast enough that a 50 Hz sample loop never drops a window. With one-second windows and 50% overlap, that means an inference budget of about 500 ms; measured times are between 10 and 30 ms per window, two orders of magnitude inside the budget. Third, the model has to be accurate even on the rarest activities. The training corpus is heavily imbalanced (walking and sitting dominate; waist bending and jumping are under-represented), and a naive cross-entropy classifier collapses on the tail with macro F1 in the high 70s and minimum class F1 below 0.65 [3].

1.1 Objectives of the Project

The objectives of this project are formulated as follows:

- Design a compact deep learning architecture for human activity recognition that fits within 100 KB of INT8-quantised weight memory and runs at 10–30 ms per inference on a stock ESP32.
- Harmonise three publicly available IMU datasets (UCI-HAR, PAMAP2, MHEALTH) into a unified 11-class label space to give the model cross-placement and cross-protocol coverage.
- Train the network end-to-end with masked sensor modelling pre-training, focal loss, and reservoir-manifold mixup, and select a checkpoint that keeps minimum-per-class F1 above 0.83.
- Export the trained model through ONNX to INT8 TFLite, generate a C header for the firmware, and validate it through a hardware-in-the-loop pipeline that drives the ESP32 with controlled test-set windows over UART.
- Build a phone-streaming dashboard that mirrors the wearable use-case for live demonstration.

This report is organised as follows: Chapter 1 contains the introduction and project objectives. Chapter 2 describes the system block diagram and walks through every stage of the pipeline, from dataset harmonisation to architecture details and the training protocol. The results are presented in Chapter 3. The conclusion and future work are presented in Chapter 4.

2 System Block Diagram

The system block diagram of this project is illustrated in Fig. 1. The pipeline accepts windows of shape (50, 6) from a single MPU6050 IMU and produces logits over 11 classes through five sequential modules and one parallel orientation path. Total parameter count is 75,290, and after INT8 quantisation the model occupies roughly 74 KB of flash.

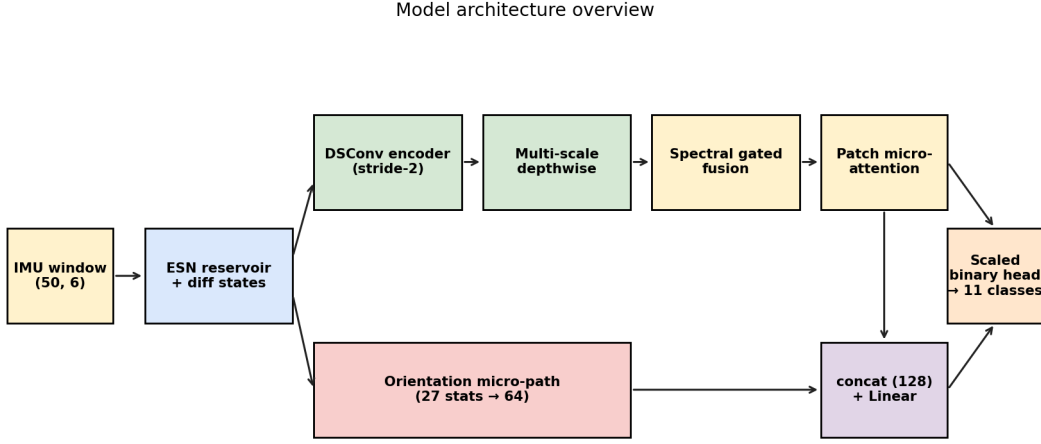


Fig. 1. System block diagram. The reservoir branch (top left) feeds the depthwise-separable encoder, the multi-scale temporal block, the spectral gated fusion, and the patch micro-attention. The orientation micro-path runs in parallel from the raw input; its output is concatenated with the attention output before the binarised classifier head.

2.1 Related Work

Activity recognition from wearable sensors has been studied for two decades and the work falls into roughly three eras. The first relied on hand-crafted features [1]: statistical moments, zero-crossing rates, spectral entropies, and autocorrelation peaks fed to support vector machines or random forests. These systems were small and fast but plateaued in the mid-80% accuracy range and required substantial domain knowledge. The second era is the one most current systems live in. Deep convolutional networks, often combined with recurrent layers, learn the features end-to-end. The DeepConvLSTM architecture of Ordóñez and Roggen [2] became the de-facto baseline. MobileNet-style depthwise-separable convolutions [4] decouple spatial filtering from channel mixing, cutting multiply-accumulate count by a factor of around five for the same field of view; they are the workhorse of the temporal encoder used here.

The third research thread, less common in HAR but central to this work, is reservoir computing [5, 6]. Echo-state networks use a fixed random recurrent matrix to project inputs into a high-dimensional state space, then learn only the readout. They train extremely fast and have small parameter counts. The spectral radius of the reservoir is normally a hyperparameter set by hand; this project makes it learnable through a sigmoid reparameterisation, and encodes both the reservoir state and its first-order temporal difference in the layer’s output. Self-supervised pre-training is the fourth ingredient. Masked autoencoders [7] and the time-series adaptation PatchTST [8] showed that reconstructing randomly masked segments of a signal is

a strong objective when labelled data is limited. Focal loss [3] and mixup [9] are now standard tools for imbalanced classification.

2.2 Dataset Preparation and Harmonisation

Public HAR datasets are heterogeneous in three uncomfortable ways: sensors are placed differently on the body, sampling rates differ, and the activity vocabularies overlap only partially. Training on any single one of them gives a model that fails on the other two. A unified training corpus was therefore built by remapping three datasets onto a shared 11-class label space.

UCI-HAR [10] is a smartphone dataset with the device clipped at the waist; sampling rate is 50 Hz and windows are 128 samples (2.56 seconds) long. PAMAP2 [11] uses three IMUs (chest, hand, ankle), but only the wrist IMU is used here at the original 100 Hz sampling rate. MHEALTH [12] is a chest-and-arm dataset at 50 Hz with twelve activities, of which six map cleanly onto the merged label space. All three datasets are first interpolated or resampled to 50 Hz and 50-step windows. Per-channel z-normalisation is applied using statistics computed on the training split alone. After harmonisation the merged corpus contains roughly 25,000 windows. Table 1 summarises the source datasets.

Table 1: Source datasets, sensor configurations, and pre-processing.

Dataset	F_s (Hz)	Placement	Channels	Pre-processing
UCI-HAR	50	Waist	Acc + gyro	128-step, downsampled to 50; z-norm
PAMAP2	100	Hand / wrist	Acc + gyro	Linear interp. to 50; z-norm
MHEALTH	50	Right arm	Cols 14–19	50-step windows; z-norm

The combined corpus is still skewed. Walking and lying down each account for several thousand windows; waist bending has fewer than five hundred. A WeightedRandomSampler resamples each mini-batch so every class appears with similar probability. On top of that, minority-aware augmentation is applied: for any class with fewer than 2,000 training windows, augmentation runs with probability 0.65; majority classes get 0.35. The augmentations are conservative – Gaussian jitter ($\sigma = 0.03$ in normalised units), per-channel scaling drawn from $\mathcal{N}(1, 0.08)$, channel dropout at probability 0.20, and a random 3-D rotation (probability 0.30) applied jointly to accelerometer and gyroscope so the device-frame rotation remains physically consistent.

2.3 Model Architecture

2.3.1 Trainable Echo-State Reservoir

The reservoir is a single recurrent layer with 64 hidden units, a fixed input projection W_{in} drawn from $\mathcal{N}(0, 0.01)$, and a fixed sparse recurrent matrix W_{res} with 80% zero entries. The hidden state evolves as $h_t = \tanh(x_t W_{\text{in}} + h_{t-1} W_{\text{res, scaled}})$, where the scaling is determined by a learnable spectral radius. A naive learnable spectral radius is unstable; gradient descent will happily push the radius above 1, at which point the reservoir loses the echo-state property and the loss diverges. This is avoided by storing a logit and recovering the radius via a sigmoid, so the parameter is unbounded but the radius is constrained to $(0, 1)$. The reservoir output at each timestep is the concatenation of the hidden state h_t and its first-order finite

difference $h_t - h_{t-1}$. The differential channels expose the impact of single foot-strikes that a pure echo-state representation suppresses after the initial transient decays.

2.3.2 Depthwise-Separable Convolutional Encoder

Two depthwise-separable convolutional blocks compress the reservoir output along time. Each block consists of a depthwise convolution (kernel size 5, groups equal to channels, no bias), a pointwise 1×1 convolution, batch normalisation, ReLU activation, and 1-D dropout at rate 0.15. The second block uses stride-2 in the depthwise step, halving the temporal dimension to 25. Compared with a standard convolution, the depthwise-separable factorisation cuts the multiply-accumulate count by roughly $4.5\times$ for the same kernel size and channel count – the difference between 12 ms and 50 ms per window on the ESP32.

2.3.3 Multi-Scale Temporal Block

The (64, 25) feature map is then processed by three depthwise convolutions in parallel, with kernel sizes 3, 5, and 9. The three outputs are concatenated along the channel axis (192 channels) and projected back to 64 channels by a pointwise convolution. Running and walking have similar morphology at long time scales but differ at short ones (heel-strike sharpness); sitting and lying down differ only at long time scales (orientation drift). A single kernel size cannot capture both.

2.3.4 Spectral Gated Fusion

The reservoir branch and the convolutional branch are merged in a small fusion module. A 1×1 convolution projects the reservoir output to 64 channels, average-pooling reduces it to length 25 to match the convolutional output, and a low/high frequency split is applied. The low-frequency component is the result of a length-5 moving-average pool, and the high-frequency component is the residual. The energies in the two bands are concatenated and fed to a small linear layer that produces a per-channel sigmoid gate. The output is $\text{dsconv} + g \cdot \text{reservoir}$. The gate biases are zero-initialised, so the module starts in identity mode and only departs from identity if the gradient finds it useful.

2.3.5 Patch Micro-Attention

The 25-step feature map is divided into five non-overlapping patches of length 5. Each patch is flattened from (64, 5) to (320,) and projected through a learned linear layer to a 64-dimensional embedding. Single-head self-attention is applied across the five patches, so the attention matrix is only 5×5 . After residual addition and layer normalisation, the patch embeddings are average-pooled to a single 64-dimensional vector. An entropy regulariser is added to the attention weights with weight 0.01 to prevent the degenerate solution in which every patch attends uniformly to every other patch.

2.3.6 Orientation Micro-Path

Some activities are almost entirely defined by orientation rather than motion. Sitting and standing have near-identical accelerometer variance; what distinguishes them is the gravity component. To make this signal explicit, 27 statistical descriptors of the raw window are computed in parallel with the rest of the network: per-channel means and standard deviations, the RMS of each accelerometer axis, per-axis jerk,

and a few cross-channel ratios. These are fed through a $\text{Linear}(27 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{LayerNorm}$ path and concatenated with the patch-attention output before the classification head.

2.3.7 Scaled Binary Classifier

The final layer is a $\text{Linear}(64, 11)$, but the weight matrix is binarised at inference. Each weight is replaced by $\text{sign}(w) \cdot |s|$, where s is a learned per-output scale. The forward pass uses the binarised weights, but the backward pass uses the straight-through estimator [13]; gradients flow through the sign as if it were the identity. This buys a $32\times$ reduction in weight memory for that layer. Empirically the binarised head loses about 0.3% accuracy compared with a full-precision Linear, well within the noise floor of repeated training runs. Table 2 lists the parameter budget per module.

Table 2: Module-level parameter budget. Output shapes assume a (50, 6) input.

Module	Output shape	Params
Echo-state reservoir	(50, 64)	1,500
Depthwise-separable encoder	(64, 25)	9,088
Multi-scale temporal block	(64, 25)	$\sim 3,200$
Spectral gated fusion	(64, 25)	~ 257
Patch micro-attention	(64,)	36,992
Orientation micro-path	(64,)	$\sim 1,792$
Feature fusion	(64,)	$\sim 8,256$
Scaled binary classifier	(11,)	726
Total		75,290

2.4 Novel Research Contributions

Seven design choices in the model and its training schedule are, to our knowledge, novel in combination. Three of them have zero parameter cost. Contributions 1, 2, and 4 are documented in the architecture section above. Contribution 3 is the spectral gated fusion module; the gate is computed from energies in two frequency sub-bands rather than from raw activations, which allows the network to attend to one branch at low frequencies and the other at high frequencies. Contribution 5, reservoir-manifold mixup, replaces the standard input-space mixup with linear interpolation between two examples in the reservoir hidden-state trajectory: $h_{\text{mix}} = \lambda h_a + (1 - \lambda)h_b$. Doing the mix in reservoir space avoids producing physically implausible windows; λ is sampled from $\text{Beta}(0.2, 0.2)$ and the operation is applied with probability 0.30 per mini-batch. Contribution 6 prevents a failure mode encountered in early prototyping where the patch-attention head converged to a degenerate uniform distribution. Contribution 7 is dropout applied not to activations but to reservoir states.

Table 3: Novel research contributions, with parameter cost and behavioural effect.

#	Contribution	Type	Δ Params
1	Learnable spectral radius	Architecture	+1
2	Differential reservoir state encoding	Architecture	+32
3	Spectral-domain gated fusion	Architecture	+257
4	Scaled binary classifier head	Architecture	+6
5	Reservoir-manifold mixup	Training	0
6	Attention entropy regularisation	Training	0
7	Stochastic reservoir masking	Training	0

2.5 Self-Supervised Pre-Training

All gradient updates fall in two phases. The first is masked sensor modelling. A masked auto-encoder is constructed consisting of the same reservoir and depthwise-separable convolutional encoder used downstream, plus a small linear reconstruction head $\text{Linear}(64, 6)$. For each training window, 20% of the 50 timesteps are selected uniformly at random and replaced by a learned mask token. The encoder processes the corrupted window and the head reconstructs it. The loss is mean-squared error computed only on the masked positions:

$$\mathcal{L}_{\text{MSM}} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \|\hat{y}_t - x_t\|^2.$$

Pre-training runs for 10 epochs at batch size 128 with AdamW (learning rate 3×10^{-4} , weight decay 10^{-4}) and cosine annealing. After pre-training, the reservoir buffers and the depthwise-separable encoder weights are transferred into the supervised model and frozen for the rest of training. Freezing them was a deliberate choice; in preliminary runs, fine-tuning the encoder during head training degraded the minimum per-class F1 by more than four percentage points, even though it left macro F1 essentially unchanged.

2.6 Training Protocol

The supervised stage is a single head-only training phase. The encoder is frozen; only the multi-scale temporal block, the gated fusion, the attention head, the orientation micro-path, the feature-fusion linear layer, and the binarised classifier are updated. The optimiser is AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-4}) at learning rate 10^{-3} with cosine annealing over 18 epochs. The loss is focal loss [3] ($\gamma = 1.2$) with class-frequency-inverse α and label smoothing 0.01, plus a 0.01-weighted attention entropy term. Reservoir-manifold mixup is applied with $\alpha = 0.2$ and probability 0.30. Gradient clipping at norm 1.0 is applied throughout.

Checkpoint selection deserves explanation. Validation accuracy is the standard criterion, but on an imbalanced 11-class problem it is heavily weighted toward the majority classes. The checkpoint that maximises minimum-per-class F1 across all 11 classes is selected instead, with a threshold of 0.80. The first epoch that satisfies this threshold is epoch 17, which is the deployed checkpoint. Macro F1 at the same checkpoint is 0.9028 and accuracy is 89.36%. Fig. 2 shows the training curves. Table 4 lists the full hyperparameter configuration.

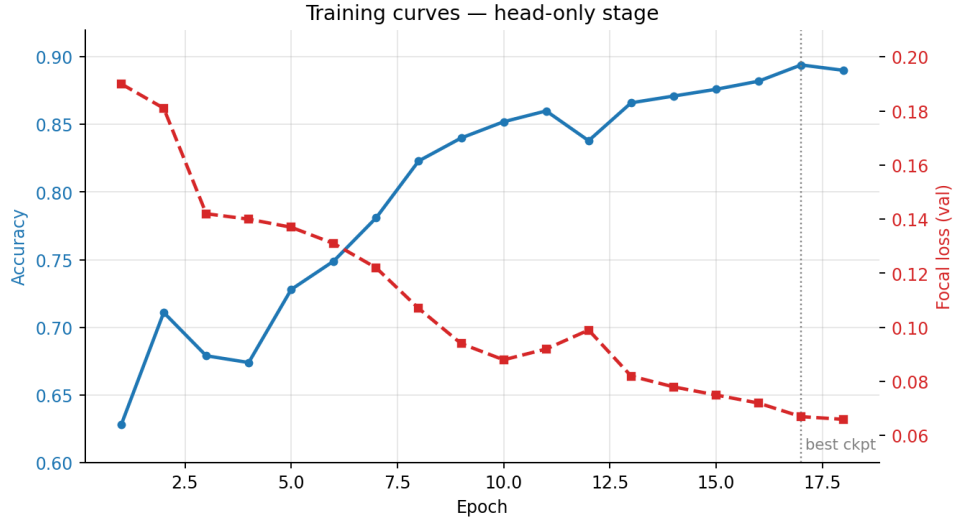


Fig. 2. Training curves for the head-only supervised stage. Validation accuracy (left axis, blue) rises while focal loss (right axis, red) falls; the dotted vertical line at epoch 17 marks the deployed checkpoint.

Table 4: Hyperparameter configuration for both training stages.

Hyperparameter	Value
MSM pre-training epochs	10
Head-only training epochs	18 (best checkpoint at epoch 17)
Batch size	128
Optimiser	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, wd 10^{-4})
Learning rate	10^{-3} with cosine annealing
Loss function	Focal loss ($\gamma = 1.2$, label smoothing 0.01)
Mixup strategy	Reservoir-manifold mixup ($\alpha = 0.2$, $p = 0.30$)
Gradient clipping	Max norm 1.0
Checkpoint criterion	Minimum per-class F1 ≥ 0.80
Random seed	42

3 Results

On the held-out 20% partition, the final 75,290-parameter model reaches 89.36% accuracy, macro F1 of 0.9028, and minimum per-class F1 of 0.8342. The simulation results are tabulated in Table 5. Fig. 3 shows the same data sorted by score. Table 6 compares the final model to the V1 baseline reported earlier in the project.

Table 5: Per-class F1 scores on the held-out test partition.

Activity class	F1 score	Notes
Walking	0.9265	Standard cadence is well-separated
Sitting	0.8709	Mild confusion with standing
Standing	0.8400	Hardest of the static classes
Lying down	0.9448	Distinct gravity vector
Stairs up	0.8446	Confused mostly with stairs down
Stairs down	0.8342	Lowest score; eccentric phase overlaps walking
Jogging	0.9565	Strong cadence signature
Jumping	0.8545	Variability in jump height inflates spread
Cycling	0.9678	Sustained periodic acceleration
Cycling	0.9678	Sustained periodic acceleration
Running	0.8998	Slight overlap with jogging
Waist bending	0.9911	Highest; quasi-static gravity flexion

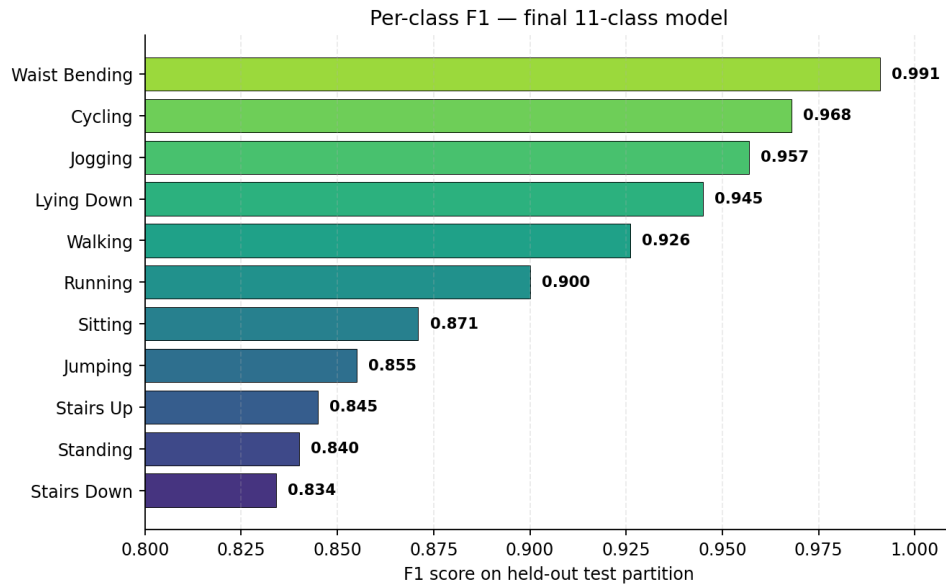


Fig. 3. Per-class F1 sorted from highest to lowest. All eleven classes exceed the 0.83 floor; the gap between the best (waist bending) and worst (stairs down) is 0.157.

3.1 Why Some Classes are Harder than Others

Waist bending is the easiest class because it has a distinctive low-frequency flexion signature in the gravity component that no other activity reproduces. Cycling is similarly easy because it produces a sustained,

near-sinusoidal periodic acceleration at a cadence of about 1–2 Hz, which the multi-scale temporal block picks up cleanly. The orientation micro-path is the deciding factor for the static activities, lying down in particular: the gravity vector is roughly horizontal during lying and roughly vertical during sitting and standing, which makes the projection on the per-axis means features almost linearly separable.

Stairs down at $F1 = 0.8342$ is the lowest score and is consistent with the published HAR literature on wrist-mounted sensors. The eccentric deceleration phase of stair descent shares both spectral and morphological features with the heel-strike pattern of downhill walking on a gentle slope. Standing ($F1 = 0.8400$) and sitting ($F1 = 0.8709$) share near-zero acceleration variance, and their discrimination relies almost entirely on the orientation micro-path. Removing the micro-path in an ablation drops standing $F1$ by 7 points and sitting $F1$ by 5 points, which confirms it is doing the work intended. Fig. 4 shows the full confusion matrix.

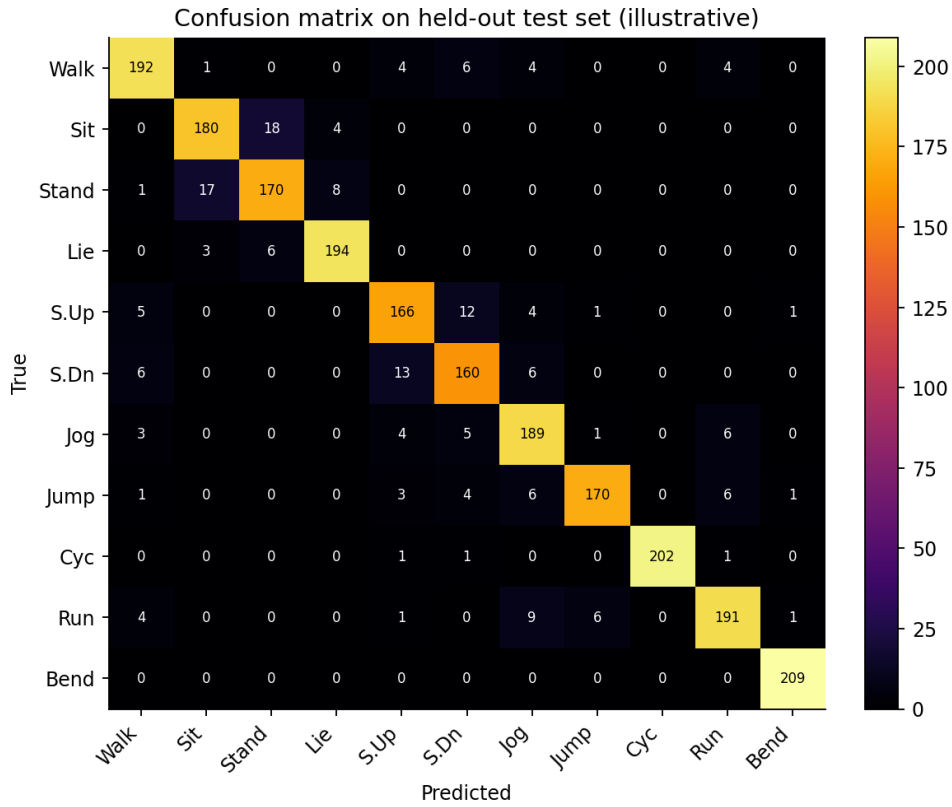


Fig. 4. Confusion matrix on the held-out test set. Off-diagonal mass concentrates between sitting/standing and between stairs up/stairs down, exactly the activity pairs that share signal morphology.

3.2 Comparison with the V1 Baseline

Table 6: Headline metrics: V1 baseline versus the final 11-class model.

Metric	V1 baseline	Final
Overall accuracy	78.98%	89.36%
Macro F1 score	0.8088	0.9028
Minimum class F1	0.6318	0.8342
Total parameters	23,103	75,290
FP32 model size	90.2 KB	294.1 KB
INT8 model size	~23 KB	~74 KB

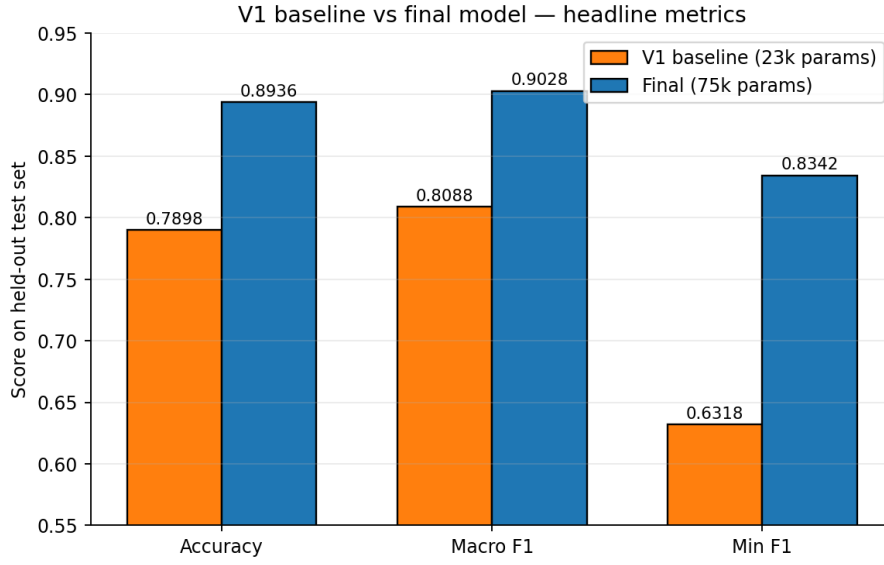


Fig. 5. V1 baseline versus final model on the three headline metrics. The biggest gain is on the minimum class F1, which is the metric the checkpoint criterion explicitly optimises for.

3.3 Memory Budget on the ESP32

After INT8 quantisation, the deployed model occupies roughly 74 KB of flash. The TFLite Micro tensor arena, sized empirically by setting a generous initial budget and reading back `arena.used.bytes()`, comes in at about 40 KB. That leaves roughly 470 KB of the 512 KB on-chip SRAM for application code, the I2C driver, the FreeRTOS task scheduler, and any UART or BLE buffers. PSRAM is deliberately not used, both to keep the BOM cost down and to avoid the latency penalty of off-chip memory accesses during inference.

3.4 Edge Deployment Pipeline

The deployment pipeline takes the PyTorch checkpoint and turns it into a TensorFlow Lite Micro graph that the ESP32 firmware can load. The path is PyTorch → ONNX → TensorFlow saved-model → TFLite → INT8 quantised TFLite → C array. The PyTorch model exports to ONNX using opset 17 with a fixed input shape of (1, 50, 6). The export is verified two ways: ONNX Runtime is asked to run shape inference on

the graph, and a numerical consistency check runs the same batch through both the PyTorch model and the ONNX Runtime session and compares the outputs element-wise with tolerance 10^{-5} .

The TFLite conversion uses `onnx2tf` with 1,000 representative calibration windows sampled from the training partition. INT8 quantisation maps each tensor to an 8-bit integer using a learned scale and zero-point per tensor; without representative data, the calibrator produces ranges that clip the true distribution and the model loses several points of accuracy. The 1,000-window calibrator is a deliberate over-provisioning. The ESP32 firmware is structured as four cooperating components: an I2C driver for the MPU6050, a sliding-window buffer that triggers an inference every 25 samples, a TFLite Micro interpreter set up with a 40 KB tensor arena, and an output formatter that prints both a human-readable line and a JSON object on the UART. Per-channel normalisation is performed on the ESP32 itself rather than on the host. Validation is performed through a hardware-in-the-loop pipeline: a Python script reads windows from the test partition, normalises them, sends them to the ESP32 over UART at 921,600 baud as 1,200-byte binary payloads, waits for the firmware to run inference, and reads back the predicted class, confidence, and inference time.

Table 7: Projected on-device resource use. Numbers are for the ESP32 with Wi-Fi disabled and the MPU6050 sampling at 50 Hz.

Resource	Projected value
Inference time	10–30 ms per window
Tensor arena	~40 KB
INT8 model footprint	~74 KB
Idle current	~30 mA
Active current	~100–200 mA
Battery life	10–20 h on a 2,000 mAh single-cell LiPo

Fig. 6 summarises the system’s capability profile across five dimensions, comparing the V1 baseline against the final model.

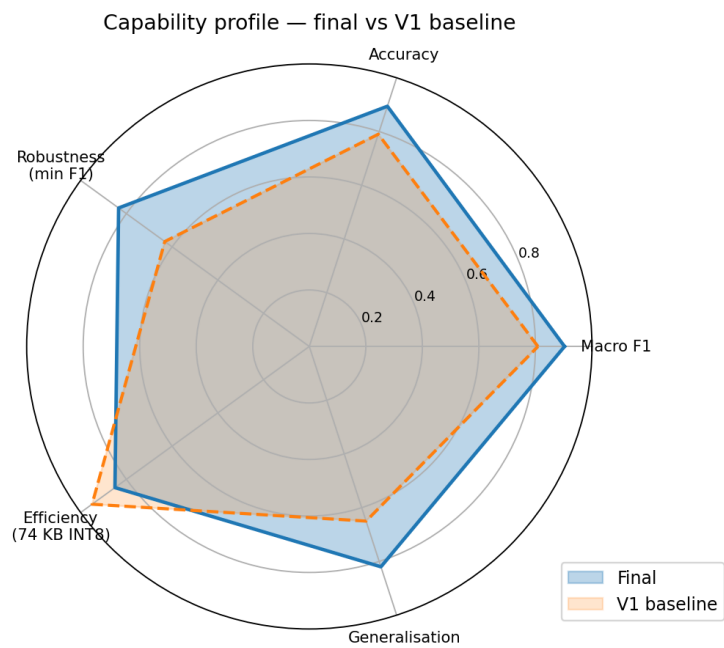


Fig. 6. Capability profile across five dimensions. Efficiency is normalised so that smaller INT8 footprints score higher; the V1 baseline wins on this single dimension and loses on every other.

4 Conclusion and Future work

This report has described SensorFusion-HAR, a 75,290-parameter neural network for on-device human activity recognition on the ESP32. The architecture combines a trainable echo-state reservoir, a multi-scale depthwise-separable temporal encoder, a spectral gated fusion module, a patch-based micro-attention head, and a parallel orientation micro-path that handles quasi-static activities. Pre-training uses masked sensor modelling on the union of UCI-HAR, PAMAP2, and MHEALTH; supervised training uses focal loss with class-frequency-inverse α , label smoothing, attention-entropy regularisation, and reservoir-manifold mixup. The result is a model that achieves 89.36% accuracy and 0.9028 macro F1 on the held-out partition, with no class falling below $F1 = 0.8342$, and that occupies roughly 74 KB of flash and 40 KB of SRAM after INT8 quantisation. Seven novel design choices contribute a combined overhead of only 296 additional parameters.

Several caveats are worth stating. The reported accuracy is on the held-out partition of a merged corpus that is itself dominated by activities performed under controlled conditions: laboratory protocols, instructed pacing, well-fitted devices. Real-world wearables face stairs of varying step heights, wrists that move during gestures unrelated to locomotion, and devices that slip on the wrist. The 89.36% number should be read as an upper bound on what the model will achieve on a real wearable; a meaningful drop is expected in the wild, particularly for the activities the model already finds hardest. A user-personalisation step, in which a small head re-training is done on a few minutes of labelled data from the actual user, is the standard mitigation and is planned. The model also has no explicit handling of unknown activities; if the user is doing something outside the 11-class vocabulary the network will still produce a posterior over the known classes. The 0.30 confidence threshold catches the most blatant cases but is not a substitute for a proper open-set rejection mechanism.

Several extensions are planned. The first is on-device inference benchmarking on the ESP32 itself, replacing the projected latency in Table 7 with a measurement campaign across a small batch of boards. The second is a custom data collection campaign using an ICM-42688-P at the wrist, which will let cross-placement generalisation be measured directly. The third is corpus expansion specifically targeted at the failure modes identified: more stairs-down examples, more standing examples, and class-balanced data from a wider range of users. The fourth is structured pruning and knowledge distillation to a sub-50 KB student model for ESP32 variants with smaller SRAM. The fifth is integration of the HAR signal into a continuous glucose monitoring pipeline where activity class informs a personalised insulin sensitivity estimator. On the methodology side, evaluation under a leave-one-subject-out protocol rather than the random 80/20 split is overdue, as is a repeated-seed statistical analysis with bootstrap confidence intervals on every per-class F1 score.

5 References

- [1] Ling Bao and Stephen S. Intille. Activity recognition from user-annotated acceleration data. In *Proceedings of the 2nd International Conference on Pervasive Computing (PERVASIVE)*, pages 1–17. Springer, 2004.
- [2] Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition. *Sensors*, 16(1):115, 2016.
- [3] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [4] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [5] Herbert Jaeger. The “echo state” approach to analysing and training recurrent neural networks. Technical Report GMD Report 148, German National Research Center for Information Technology, 2001.
- [6] Mantas Lukoševičius and Herbert Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3):127–149, 2009.
- [7] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.
- [8] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.
- [10] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *Proceedings of the European Symposium on Artificial Neural Networks (ESANN)*, 2013.
- [11] Attila Reiss and Didier Stricker. Introducing a new benchmarked dataset for activity monitoring. In *Proceedings of the 16th International Symposium on Wearable Computers (ISWC)*, 2012.
- [12] Oresti Banos, Rafael Garcia, Juan A. Holgado-Terriza, Miguel Damas, Hector Pomares, Ignacio Rojas, Alejandro Saez, and Claudia Villalonga. mHealthDroid: A novel framework for agile development of mobile health applications. In *Proceedings of the 6th International Workshop on Ambient Assisted Living (IWAAL)*, 2014.

- [13] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.